Bounded Operational Continuity in ESS-MAI: A Source-Derived PD Checkpoint and Replay Experiment

Bledar Gjata · Independent Researcher, Albania ORCID: 0009-0003-7854-266X

Research preprint, 24 October 2026. Manuscript: 10.5281/zenodo.23613785. Software artifact: 10.5281/zenodo.23613613.

Abstract

This experiment asks whether a bounded operational dialogue state can continue across process boundaries without changing its canonical checkpoint representation. AIGovern-Cyber POC008 extracts the actual Progressive Debatic (PD) core, associated data types and checkpoint machinery, then connects them to a new, small append-only storage adapter. The comparison uses identical scoped inputs and explicit time values, with uninterrupted engine execution as the control. The target is exact checkpoint-byte equality after every observed transition, rather than agreement only at the final turn. The adapter restores the current checkpoint into a fresh engine, recomputes one input, validates the resulting request, appends and synchronizes its record, then publishes the in-memory head. Replay applies the same transition checks. All 35 tests passed: 20 inherited unit tests and 15 integration tests, with none failed or ignored. The experiment concerns finite operational continuity. It does not establish scientific truth, native knowledge admission, indefinite persistence, authenticated freshness or arbitrary power-loss safety.

1. Research question and context

Retaining text is insufficient to show that a stateful mechanism continues consistently: counters, budgets, derived signals and session identity may diverge even when visible dialogue looks unchanged. POC008 therefore compares serialized engine state, using the same extracted transition implementation on both sides.

Checkpointing, journaling and replay are established engineering techniques. SQLite's account of atomic commit explicitly connects recovery guarantees to storage assumptions and crash testing; that account describes rollback-journal mode, not this capsule's log format 1. Rust documents platform-dependent interactions between file locks and ordinary reads or writes, and describes synchronization as an attempt to flush operating-system file state 2. These sources inform the experiment's limits, not a claim that POC008 reproduces SQLite's guarantees. SHA-256 supplies content digests using an established standardized algorithm 3.

The contribution offered for review is a narrowly inspectable extraction, adapter and falsifiable comparison, without a broad novelty claim. Earlier POC007 software 4 and its accompanying preprint 5 provide ESS-MAI context; their results do not validate POC008.

2. Formal comparison

Let S_0 be an initialized PD state for a fixed scope, i_n the next user input and t_n its explicit time value. Let F denote the extracted engine transition, C canonical checkpoint serialization, and R restoration into a fresh instance. Define the uninterrupted sequence by

```
S_n = F(S_(n-1), i_n, t_n).
```

For each observed step n, the target is

```
C(S_n) = C(F(R(C(S_(n-1))), i_n, t_n)).
```

The executable comparison also follows the actual restarted history: its previous accepted checkpoint is restored before the next input. After each prefix, that path's checkpoint must equal C(S_n) byte for byte. Initialization, scope, input order, timestamps and source version must match. The capsule's fixed user-only checkpoint prefix is included consistently on both paths.

Digest equality is useful for command-line reporting but is not the stipulated direct-byte assertion. Successful examples support the property only for the observed sequences, configurations and implementation. They do not prove it for all possible inputs, all concurrent schedules, infinite horizons or future versions. Both paths share the engine implementation, so a shared semantic defect could survive this comparison.

3. Artifact and commit boundary

The public package contains selected PD core, genius-detector, checkpoint and data-type code; the restricted operational PDC1 contract and PDR1 codecs; the source-derived `PdProjection`; and selected CRC framing code. Extraction provenance records source hashes, selected ranges, transformations and output hashes. A source-derived module is distinguishable from an unchanged source copy.

`Store` in `src/continuity.rs` is newly written POC code. Its `OPL2SPD1` file magic separates this format from the original KnowledgeVault. It does not implement the complete original vault or promise WAL compatibility. Each frame contains a length, its inverse, CRC and encoded record. A nonblocking exclusive file lock protects the adapter's file ownership.

An exact retry is resolved before comparison with the current head. Reusing an operation identifier with changed input or timestamp is a conflict. A new request is checked against its expected revision and predecessor, then independently reconstructed within this process by restoring the accepted checkpoint into fresh PD and ingesting exactly one input. The recomputed request must match the supplied request. This is local recomputation, not independent institutional verification.

The adapter appends the frame and calls `sync_all` before publishing the projection. An error leaves that store instance poisoned and requires reopening. Replay validates complete frames, decodes records, recomputes transitions and checks the chain. Only after all complete records validate may an eligible incomplete tail be truncated. Detectable corruption is rejected without repair.

4. Experimental method

The first fixture supplies five dialogue turns about a flag, including punctuation and Albanian Unicode. Each turn runs in a new operating-system process. The test inspects the preserved user text, revision and absence of a knowledge verdict, then reopens the store and compares its checkpoint directly with uninterrupted execution of the same prefix. These are dialogue fixtures, not a participant study or factual evaluation of the description.

A second fixture runs 65 synthetic, same-topic turns through separate subprocesses. After each step, it reopens the store and compares the actual checkpoint bytes directly with uninterrupted execution of the matching prefix. It also checks the reported SHA-256 digest. The final test passed all 65 per-step byte comparisons. No timing or throughput advantage is claimed.

Additional integration cases cover old exact retries after the head advances, changed retries, stale requests, two isolated contexts, caller-modified checkpoints and questions, malformed framing, CRC corruption, recomputed public hashes around an invalid transition, incomplete tails and wrong-parent records. A

child-process lock probe checks refusal of a competing opener and release after the lock holder is killed. That probe does not interrupt a commit. Oversized input and inherited malformed-checkpoint tests exercise bounded rejection. The 4096-turn ceiling is explicit in validation code; a direct 4096/4097 boundary test was not performed.

Validation used Rust 1.98.0 on `x86_64-pc-windows-gnu`, with warnings denied. The reproduction commands are `cargo test --locked` and `cargo build --release --locked`; the recorded release build also used `--offline` with dependencies already cached. GitHub CI run 40079957984 also passed all 35 tests and release builds on Windows and Linux using Rust 1.98.0. The job record is included in evidence/github-ci.json; these are maintainer-controlled automated runs, not independent human review. A tested minimum supported Rust version has not been established.

5. Measured results

The final recorded run passed all 35 tests, with zero failures and zero ignored tests. The unit suite completed in 0.23 seconds and the integration suite in 135.34 seconds. Direct checkpoint-byte equality held at all five dialogue prefixes and all 65 synthetic prefixes. The optimized release build completed successfully in 13.55 seconds. These are observed run durations, not controlled performance benchmarks. The supporting logs are named `evidence/windows-gnu/cargo-test.log` and `evidence/windows-gnu/cargo-release.log`.

Development exposed a partial-header defect: the initial adapter treated every short trailing header as recoverable, even when available bytes already contradicted valid framing. A failing regression, recorded in `evidence/windows-gnu/red-partial-header.log`, demonstrated this behavior. The correction checks a present length and available inverse-length bytes before accepting truncation. That regression passed in the final suite.

The first full integration run also encountered three test-harness failures from reading a file while its Windows lock remained held, recorded in `evidence/windows-gnu/first-integration-run.log`. The harness was corrected to close the store before independent file inspection; the production lock was retained. Those tests passed in the final run.

One passing case deliberately expects a valid older prefix to be **accepted**. It restores an earlier complete log and checks that the earlier head reappears. This records the rollback limitation; acceptance must never be reported as rollback protection.

6. Limits and threats to interpretation

The engine is bounded to 4096 turns and an 8 MiB checkpoint, with additional string and collection limits. Replaying and recomputing historical transitions also imposes costs not benchmarked here. No conclusion about indefinite or infinite continuity follows.

The auxiliary PDC1 input uses the explicit `POC008:SCIENCE_NOT_EXECUTED` marker. No science-query evaluation, KiE/PIM/Spine integration, native T1/T2 execution or native knowledge verdict occurs. Operational response hints remain derived dialogue state. Persistence grants no knowledge authority, and this extraction is not a one-to-one reproduction of the entire private runtime.

CRC and unkeyed hashes are not authentication. Without an external trusted freshness anchor, a valid old history can replace a newer one. The capsule cannot establish author identity, resist a hostile host or secure a compromised executable. File-lock behavior depends on the operating system and filesystem 2.

The transition witness hashes `include_bytes!("continuity.rs")`. Changing those source bytes changes the expected witness, so the revised adapter can reject an older WAL during recomputation. Compatibility is pinned to the source version; there is no migration mechanism or cross-version promise.

Subprocess relaunch and prepared malformed files do not simulate every interruption during a write. There has been no physical power-cut experiment or injected write/`fsync` failure campaign. Synchronization calls alone do not establish that storage honors every durability assumption 1, 2. Certification, AI Act compliance and external endorsement are outside the claims.

7. Authorship, availability and review

Bledar Gjata directs the architecture, research question and authority boundaries as an independent researcher. OpenAI Codex assisted source inspection, extraction planning, adapter implementation, tests and documentation, including this manuscript. An exact model-build identifier is not recorded here. The author reports reviewing generated work through shell-based checks. That statement is distinct from a claim that every command was manually typed or every line independently audited. AI agreement is not independent human review.

The Apache-2.0 package targets release `poc008-public-v0.1.0` in AIGovern-Cyber-POC008-Continuity. Its software artifact is identified by 10.5281/zenodo.23613613. Reviewers are invited to reproduce a differing checkpoint prefix, defeat the request/head checks, find a malformed tail wrongly repaired, or identify a misleading authority claim. Reports should include the exact version, platform, input bytes, ticks, expected behavior and smallest reproducer. Negative findings and the accepted rollback limitation belong in the public record.

References

1. SQLite Development Team. Atomic Commit In SQLite. Official documentation. Accessed 24 October 2026.

2. The Rust Project Developers. std::fs::File: `try_lock` and `sync_all`. Official documentation. Accessed 24 October 2026.

3. National Institute of Standards and Technology. Secure Hash Standard (SHS). FIPS 180-4, August 2015. doi:10.6028/NIST.FIPS.180-4.

4. Gjata, B. AIGovern-Cyber POC007: Obligation Promise Law to Success - public reference capsule. Version 0.1.2, 2026. doi:10.5281/zenodo.23596907.

5. Gjata, B. Constitutional Self-Governance in ESS-MAI: Evidence-Bound Authority and an OPL2S Reference Capsule. Preprint, 2026. doi:10.5281/zenodo.23597026.